

— PÉ PEDAL BENTINHO 2026

Da câmera ao *certificado*

Como o site captura, processa e entrega um registro personalizado usando apenas APIs nativas do navegador.

HTML5 + CSS3

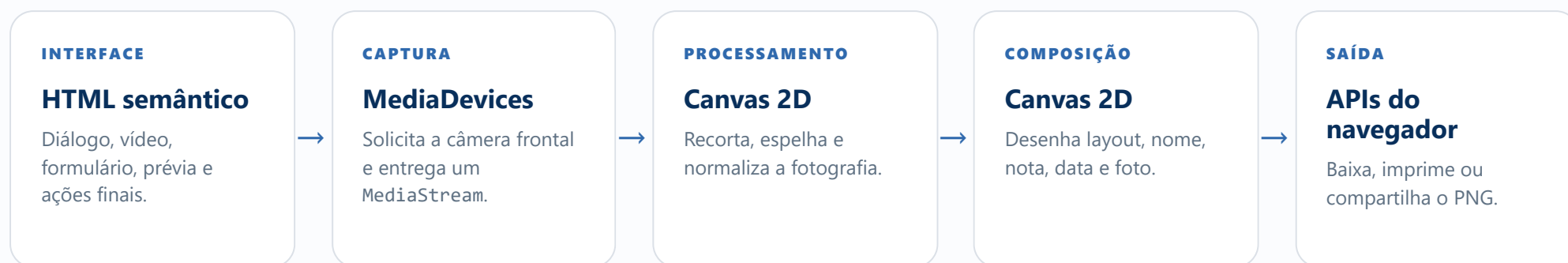
JavaScript nativo

Sem backend

Câmera → Canvas → PNG

Uma aplicação local, simples e rastreável

Toda a experiência acontece no dispositivo do participante. O navegador coordena a permissão da câmera, a composição gráfica e a exportação do arquivo.



0 requisições para um servidor durante a criação	900×1200 resolução do Canvas intermediário da foto	1600×1131 resolução do certificado final	PNG formato final para preservar texto e detalhes
--	--	--	---

Fluxo completo em seis etapas

O processo é guiado por estados de interface: cada etapa só libera a próxima quando os dados necessários estão disponíveis e válidos.

01

Abrir

O botão exibe um <dialog> modal e inicia a solicitação da câmera.

02

Autorizar

O navegador mostra a permissão. O site não consegue ignorar essa decisão.

03

Capturar

O quadro do vídeo é desenhado no Canvas ou uma foto do aparelho é carregada.

04

Confirmar

A pessoa revisa a foto, informa nome e nota e confirma a presença do rosto.

05

Gerar

O certificado é composto pixel a pixel no Canvas, inteiramente em memória.

06

Usar

O resultado pode ser impresso, baixado em PNG ou compartilhado.

Plano B: se a câmera falhar ou for bloqueada, o fluxo continua pela seleção de uma imagem já existente no aparelho.

Permissão explícita e contexto seguro

```
const stream = await
navigator.mediaDevices.getUserMedia({
  audio: false,
  video: {
    facingMode: "user",
    width: { ideal: 1280 },
    height: { ideal: 1920 }
  }
});

video.srcObject = stream;
await video.play();
```

Pré-requisito: funciona em **HTTPS** ou **localhost**. Em uma página insegura, a API é bloqueada pelo navegador.

O que cada opção significa

audio: false nenhum microfone é solicitado.

facingMode: user preferência pela câmera frontal.

ideal resolução desejada, não uma exigência rígida.

MediaStream fluxo ao vivo associado ao elemento <video>.

Ciclo de vida controlado

Ao capturar, fechar ou reiniciar, o código percorre as trilhas e chama `track.stop()`. Isso libera a câmera e apaga o indicador de uso do sistema.

Falhas previsíveis recebem respostas específicas

A interface traduz erros técnicos em orientações acionáveis e mantém uma alternativa acessível para concluir o processo.

!isSecureContext

Orientar o uso de HTTPS/localhost ou foto do aparelho.

NotAllowedError

A permissão foi negada ou bloqueada nas configurações.

NotFoundError

O dispositivo não possui câmera disponível.

NotReadableError

Outro aplicativo ou processo está usando a câmera.

NotSupportedError

A API de captura não existe nesse navegador.

InvalidFile

O arquivo escolhido não é uma imagem válida.

15 MB

Imagens acima do limite são recusadas antes do processamento.

requestId

Ignora respostas atrasadas e encerra streams que ficaram obsoletos.

Estado visível

Mensagens de “solicitando”, “pronta” e erro são anunciadas em uma região com `role="status"`.

Ação protegida

O botão de captura permanece desativado até o vídeo realmente estar pronto.

URL temporária

Na foto enviada, `URL.revokeObjectURL()` libera a referência após a leitura.

Da fonte original a uma foto consistente

A função `drawCoverImage()` aplica a mesma regra para câmera e arquivo: preencher a área inteira sem distorção, cortando apenas o excedente central.



Uma composição gráfica criada em tempo real



1. Preparação

Aguarda `document.fonts.ready`, preenche o fundo e cria gradientes.

2. Camadas vetoriais

Molduras, divisórias, logotipo tipográfico, bicicleta e caixa da nota são desenhados com paths do Canvas.

3. Dados dinâmicos

Foto, nome, nota e data em português são posicionados na arte. O tamanho do nome reduz até caber.

4. Serialização

`toDataURL("image/png")` transforma o Canvas de 1600×1131 em uma imagem exibível e baixável.

Três saídas, uma única imagem-fonte

O PNG gerado é reaproveitado em todos os caminhos. Isso evita divergências entre aquilo que a pessoa vê, baixa, imprime ou compartilha.



Download

- Cria um link temporário.
- Define href com o Data URL.
- Usa nome de arquivo normalizado, sem acentos e limitado a 45 caracteres.



Impressão

- Monta um iframe isolado.
- Configura A4 horizontal sem margens.
- Exibe o PNG com `object-fit: contain` e chama `print()`.



Compartilhamento

- Converte Canvas em Blob e depois em File.
- Testa `navigator.share` e `navigator.canShare`.
- Se indisponível, faz download automaticamente como fallback.

Compatibilidade progressiva: recursos mais avançados são usados quando existem; o download continua sendo o caminho universal.

Dados pessoais permanecem no aparelho

Modelo atual

Processamento local

A foto, o nome e o certificado vivem apenas em variáveis e Canvas na memória da página. Não existe envio para API, servidor ou banco de dados.

Câmera temporária

O stream é encerrado depois da captura e também ao fechar, refazer ou reiniciar o fluxo.

- ✓ **Consentimento do navegador:** o usuário decide se permite a câmera.
- ✓ **Sem áudio:** o site solicita somente vídeo.
- ✓ **Sem persistência:** nome e foto não entram no localStorage.
- ✓ **Sem transmissão:** não há fetch, upload ou endpoint no fluxo.
- ✓ **Confirmação adicional:** o formulário exige declarar que o rosto aparece na foto.
- ✓ **Limpeza:** ao reiniciar, variáveis, imagens e os dois Canvas são esvaziados.

Limite importante: se futuramente houver backend ou galeria pública, será necessário rever consentimento, retenção, autenticação e política de privacidade.

Demonstração técnica em aproximadamente 3 minutos

- 1 Abra o site em HTTPS ou localhost.**
Explique que a origem segura é requisito da API de câmera.

- 2 Clique em “Criar meu certificado”.**
Mostre a permissão controlada pelo navegador.

- 3 Autorize e capture a foto.**
Destaque câmera frontal, espelhamento e corte proporcional.

- 4 Preencha nome, nota e confirmação.**
Mostre a validação antes de gerar.

- 5 Gere o certificado.**
Explique que o Canvas combina elementos gráficos e dados em memória.

- 6 Teste download ou compartilhamento.**
Finalize com a ideia de compatibilidade progressiva e fallback.

Frase-chave

“O navegador funciona como câmera, editor gráfico e exportador; nenhum dado pessoal precisa sair do dispositivo.”

Teste de falha

Negue a permissão de câmera para demonstrar a mensagem específica e a alternativa de escolher uma foto.

Pontos para perguntas

HTTPS

Canvas

MediaStream

Web Share

privacidade

— CONCLUSÃO

Uma solução *sem backend*, com experiência completa

Permissão explícita, processamento local, composição gráfica no Canvas e múltiplas formas de saída — tudo com APIs nativas e fallback para diferentes dispositivos.

`getUserMedia()` → Canvas 2D → PNG → download / print / share